

Relazione SDCC-ML

Federico Di Stefano, 0376007

federico.distefano.12@alumni.uniroma2.eu University of Rome Tor Vergata

University of Rome Tor Vergata

Roma, Lazio, Italy

Cristiano Pera, 0381560

Roma, Lazio, Italy

cristiano.pera@alumni.uniroma2.eu

Alfonso Maiorca, 0382768

alfonso.maiorca@students.uniroma2.eu

University of Rome Tor Vergata

Roma, Lazio, Italy



CONTENTS

Contents	1
1 Introduzione	1
2 Sistema Decentralizzato	1
2.1 Architettura Sistema Distribuito	2
2.2 Comunicazione	3
2.3 Training Modello AI	4
3 Sistema Centralizzato	5
4 Osservazioni	5
4.1 Caso decentralizzato	6
4.2 Confronto con caso centralizzato	7
5 Conclusioni	8
References	8

1 Introduzione

La seguente relazione ha lo scopo di presentare il progetto combinato dei corsi di "Sistemi Distribuiti e Cloud Computing" e "Machine Learning" per l'anno accademico 2025/2026.

È stata richiesta la progettazione, implementazione e valutazione di un sistema per l'addestramento federato di modelli di Machine Learning in modalità decentralizzata. Il sistema deve permettere a molteplici client, ciascuno con la sua porzione di dataset locale, di collaborare all'addestramento di un modello globale senza scambiare dati degli utenti, ma solo dati del modello stesso.

Il sistema deve soddisfare i requisiti elencati di seguito:

- (1) Architettura decentralizzata la quale permetta ad un qualsiasi client di avviare un processo di training di un modello basato su reti neurali.
- (2) Il processo di addestramento deve implementare una strategia di aggregazione eseguita in maniera decentralizzata, evitando che un singolo nodo agisca come aggregatore unico.
- (3) Un sistema di fault tolerance che permetta al sistema di recuperare da eventuali guasti o errori.

(4) Inoltre deve essere condotta un'analisi su prestazioni, scalabilità ed efficienza delle comunicazioni.

(5) Infine deve essere creata una versione centralizzata del sistema da mettere a confronto con quella decentralizzata.

2 Sistema Decentralizzato

Per la realizzazione del progetto è stato implementato un sistema distribuito che tramite un service registry fornisce i riferimenti dei client che partecipano all'addestramento. Essendo una simulazione è prevista anche la creazione da parte di quest'ultimo dei nodi mancanti. L'addestramento parte quando un utente fa partire un client da AWS. Il processo di training consiste nell'alternanza tra l'addestramento locale e l'applicazione dell'algoritmo di FedAvg[2]. Nella prima fase abbiamo che per un certo numero di epoche (definito nella variabile d'ambiente NUM_EPOCHS) il client addestra il suo modello locale. Successivamente il client propaga il suo modello aggiornato agli altri nodi e rimane in attesa dei loro pesi. Infine il nodo applica la FedAvg con i dati ricevuti. Un round è definito come un'alternanza tra il training locale e l'applicazione della FedAvg. L'addestramento del client è quindi composto da un numero di round definito nella variabile d'ambiente TOTAL_ROUNDS.

Essendo applicati un certo numero di round, il sistema convergerà ad un modello globale generalizzato in cui è stato incluso almeno una volta il modello locale di ogni nodo.

Siccome il modello finale avrà avuto influenza di tutti gli altri nodi, esso includerà le informazioni da tutti i dataset locali, e sarà quindi più potente dei singoli modelli locali.

Inoltre il sistema è agnostico al modello utilizzato per il training. Ciò comporta la possibilità di usare qualsiasi tipo di modello PyTorch senza compromettere la funzionalità del sistema fornendo così un'elevata modularità. Modificando il file "model.py" è possibile cambiare il modello. Bisogna solo tenere a mente che la parte della rete neurale da addestrare deve essere contenuta in `self.fc`, perché di quella parte è impostato l'addestramento locale e lo spostamento dei pesi (con `get_weights_as_bytes` in `utils.py`).

2.1 Architettura Sistema Distribuito

2.1.1 Strumenti Utilizzati

- (1) AWS Academy Learner Lab: Versione trial di un account AWS messa a disposizione per il progetto.
- (2) VPC (Virtual Private Cloud): Rete virtuale dedicata all'account AWS.
- (3) S3 (Simple Storage Service): Servizio di archiviazione.
- (4) ECR (Elastic Container Registry): Registro per le immagini dei container.
- (5) ECS (Elastic Container Service): Container orchestration service.
- (6) DDB (DynamoDB): Database NoSQL.
- (7) CloudWatch: Strumento per monitorare eventi ed outputs.

2.1.2 Architettura AWS

Per l'architettura del sistema è stato usato AWS mediante AWS Academy Learner Lab. Come primo step si è utilizzato il servizio VPC per estrarre gli indirizzi delle subnet a nostra disposizione. Gli indirizzi così ottenuti sono poi stati fissati nel codice delle utils usate per interagire con i servizi di AWS. Il dataset è stato caricato sul servizio S3 in modo da permettere ai client di scaricarlo localmente in maniera autonoma. All'avvio del training ogni client prenderà una porzione del dataset che utilizzerà per il training mediante un algoritmo che assicura che la divisione non causi sovrapposizioni.

Si è quindi utilizzato ECR per caricare le due immagini dei container utilizzati per le task definition di ECS. Una di esse è usata per i client mentre l'altra per il service registry. Le task sono nominate nel seguente modo:

- `client_task`
image: client/Dockerfile,
ecr: federated/client
- `registry_server_task`
image: registry/Dockerfile,
ecr: federated/registry

Le task definition così create vengono impiegate in un cluster dedicato. In quest'ultimo è presente un servizio (`registry-service`) con il compito di tenere attiva un'istanza del service registry.

Grazie a un load balancer fornito da EC2, i client non hanno bisogno di sapere l'indirizzo ip specifico del registry, perché essi utilizzano sempre lo stesso indirizzo statico (`DNSname: porta`) che viene rimappato dal load balancer all'effettivo indirizzo del container, utilizzando un servizio di DNS.

Si è optato per un approccio serverless mediante l'utilizzo di Fargate. Ciò garantisce un'elevata flessibilità di costi e semplicità d'uso, ma risulta essere incompatibile con acceleratori gpu. Ciò ha portato, senza contare le numerose limitazioni poste dal Learner Lab in proposito, a rinunciare all'utilizzo dei suddetti.

Il service registry tra le altre cose si occupa di registrare i client su DynamoDB dove compare il loro ID e il loro status. In questo

modo è possibile tenere traccia di tutti i client attivi e del loro status anche in caso di un guasto del service registry. Per venire incontro a situazioni di cold start i client, una volta terminato il training, tornano idle per un lasso di tempo fisso prima di terminare. Se durante questo intervallo viene avviato un nuovo training, i nodi in stato di riposo possono quindi unirsi al nuovo training, riducendo così i tempi di attesa per creazione di nuovi client. Si è infine utilizzato CloudWatch per monitorare il sistema.

Per la versione centralizzata è stata usata un'architettura equivalente con il proprio database di DDB e cluster, contenente le seguenti task definitions:

- `client_task_centralized`
image: centralized/Dockerfile,
ecr: federated/training-centralized
- `coordinator_task`
image: centralized/Dockerfile,
ecr: federated/training-centralized
- `registry_server_task_centralized`
image: registry_centralized/Dockerfile,
ecr: federated/registry-centralized

L'unica differenza sostanziale è la presenza della `"coordinator_task"` la quale si occupa dell'avvio del training, dell'aggregazione dei pesi e della suddivisione del dataset. Anche qui è stato creato un servizio per tenere attiva un'istanza di `"registry_server_task_centralized"`, è identica a `"registry_server_task"` con l'unica differenza presente negli indirizzi.

`"coordinator_task"` e `"client_task"` condividono la stessa immagine ma hanno un punto di avvio diverso, specificato nelle impostazioni di Docker.

2.1.3 Esecuzione

Per lanciare il sistema basta creare manualmente una `"client_task"` impostando in container overrides la seguente variabile d'ambiente:

- `STARTER = true`

Modificando invece i seguenti parametri si può personalizzare il processo di addestramento:

- `NUM_EPOCHS` per cambiare il numero di epoche in un round.
- `TOTAL_ROUNDS` per il numero di round. Alla fine di ogni round avverrà lo scambio dei pesi. Concluso l'ultimo round l'addestramento termina.
- `TRAINING_NODES` per il numero di nodi coinvolti nell'addestramento.
- `NUM_PEERS_REQUIRED` numero minimo di peers per avviare il training. Può essere impostato al più a `TRAINING_NODES - 1`.
- `WEIGHT_WAIT_TIMEOUT_SECONDS` per impostare il tempo massimo di attesa per la ricezione dei pesi.

Per il sistema centralizzato il procedimento è analogo, utilizzando una `"coordinator_task"` con le variabili d'ambiente desiderate. In

questa versione STARTER non è presente in quanto il coordinatore avvia sempre il processo.

In entrambi i casi deve essere attiva una task del rispettivo service registry, il che viene garantito dai servizi descritti in precedenza.

2.1.4 Flusso di lavoro del sistema

Il flusso del sistema viene riportato di seguito. Per il sistema decentralizzato, abbiamo un solo tipo di task (client_task). Essa esegue le seguenti operazioni:

- (1) Lettura delle variabili d'ambiente, per determinare la propria modalità operativa
- (2) Avvio del server gRPC e registrazione al service registry
- (3) Diramazione in base alla variabile STARTER:
 - (a) Se la variabile STARTER è impostata a FALSE:
 Attesa in idle di una richiesta di avvio training da un nodo starter. Dopo un tempo di attesa prefissato (default: 5 minuti), shutdown
 - (b) Se la variabile STARTER è impostata a TRUE:
 Il nodo è uno starter:
 - (i) Lettura della configurazione dalle variabili di ambiente
 - (ii) Richiesta peers al service registry
 - (iii) Invio richiesta di training e configurazione a tutti i peers ricevuti
- (4) Se il nodo è stato ricreato a seguito di fault recovery (variabile RESPAWNED = TRUE):
 Recupero del modello aggiornato e passaggio al round successivo
- (5) Avvio dell'addestramento
 - (a) Download del dataset e proporzionamento locale
 - (b) Training loop:
 - (i) Training locale
 - (ii) Serializzazione del proprio modello
 - (iii) Invio pesi ai peers tramite gossiping
 - (iv) Attesa dei pesi aggiornati dai peers
 - (v) Aggregazione pesi tramite FedAvg
 - (vi) Passaggio al round successivo

Al termine dell'addestramento:

- (6) Inferenza sul test set e calcolo delle metriche delle prestazioni.
- (7) Rientro in modalità idle e attesa. Se viene ricevuta una nuova richiesta di training, ritorno al punto 5
- (8) Trascorso il tempo di attesa, deregistrazione dal service registry, shutdown server gRPC e deprovisioning del container

Per quanto riguarda il sistema centralizzato, esso è diviso in due task: coordinator e client. Il client esegue tutti i punti tranne il 3, del quale si occupa il coordinatore. La differenza è che in questo caso i pesi non vengono inviati ai peers, dei quali i client non sono a conoscenza, bensì al coordinatore centrale che, una volta ricevuti tutti, effettuerà la FedAvg e restituirà il modello aggiornato a tutti i client. Esso si occupa inoltre di effettuare la divisione del dataset e il reclutamento dei nodi, nonché di inviargli la richiesta di avvio training.

2.1.5 Interfaccia con AWS

Per la comunicazione tra registry e AWS sono state create diverse funzioni di utility scritte in go. Abbiamo AddNode e RemoveNode le quali vengono rispettivamente usate dal registry per aggiungere e rimuovere i client da DynamoDB. La utility NodeCount fornisce il numero dei client mentre FetchIdleNodes restituisce una lista contenente tutti i client registrati che risultano con status "idle". Lo status dei client può essere modificato grazie alla funzione ChangeStatus, ma solamente lo status "idle" e "working" è utilizzato nel sistema. Per creare i client viene usata la funzione LaunchTask la quale include un campo per inizializzare i parametri di environment per il container. Per finire GetFullLocalAddress restituisce l'indirizzo e la porta locale. Il registry utilizza questa funzione per fornire il suo indirizzo ai nodi creati.

2.2 Comunicazione

Per gestire la comunicazione, il sistema utilizza il framework gRPC, che permette di ridurre al minimo la latenza e l'overhead di comunicazione. Tutti gli scambi di dati avvengono quindi tramite il protocollo RPC, che permette inoltre di gestire la condivisione di informazioni tra architetture diverse senza problemi di compatibilità. Il sistema infatti, dovendo scambiare dati tra un service registry scritto in Go e i nodi scritti in Python, utilizza dei messaggi e dei servizi definiti all'interno del file federatedproto. Questo file permette di definire delle strutture dati fisse, e delle chiamate comuni tra tutte le architetture. Una volta compilato, vengono generati file diversi per le rispettive componenti, che forniscono delle interfacce contenenti tutte le strutture dati e le chiamate definite nel file proto. Una volta implementate, queste interfacce possono essere utilizzate per attuare la comunicazione, senza bisogno di aggiungere nulla di specifico.

2.2.1 Scambio dei pesi

Lo scambio di maggiore entità è sicuramente quello che riguarda i pesi dei modelli aggiornati. Questi infatti arrivano a pesare oltre 400kb, che rappresentano un carico non indifferente sulla rete, se ripetuto per ogni nodo. Questo problema è stato quindi trattato utilizzando strumenti adeguati forniti dai framework utilizzati. Il modello infatti, essendo implementato tramite PyTorch, presenta le funzioni save e load, che sono essenziali per il tipo di sistema realizzato. Queste funzioni permettono infatti di salvare lo state dict dei modelli sotto forma di flussi di byte, escludendo il transformer che genera l'embedding per la rete neurale. Alla stessa maniera, la funzione load permette di caricare questi byte su un modello, per ripristinarne lo state dict. I pesi sotto forma di byte vengono scambiati tramite gRPC, utilizzando proprio il tipo bytes, che manda i dati sotto forma di bytes semplici. Questo ha permesso di ridurre

al minimo l'overhead di comunicazione, lasciando ai nodi locali la gestione dei pesi ricevuti.

2.2.2 Protocollo di Gossip

La comunicazione tra i nodi decentralizzati è stata implementata utilizzando un protocollo di gossiping: i client non inviano i propri pesi a tutti gli altri nodi presenti nella rete, ma solo a un piccolo sottoinsieme preso randomicamente dalla lista dei peers. Il numero dei nodi è calcolato in questa maniera:

$$k = \lceil \ln(N) \rceil + c$$

Dove

$$N = |P| + 1$$

Rappresenta il numero di peers + 1 e c è una costante di ridondanza, aggiunta per assicurare dell'overlap nel gossiping. Il numero di nodi a cui vengono inviati i propri pesi è quindi molto piccolo rispetto al numero totale, ma questo non rappresenta un problema, in quanto dalla teoria del gossiping entro la fine dell'addestramento ogni nodo avrà visto ogni altro modello nella rete.

Prima di proseguire all'aggregazione dei pesi ricevuti, ogni nodo aspetta un timeout prefissato. Durante questo periodo, aspetta di ricevere almeno un numero di pesi pari a

$$\left\lceil \frac{k}{2} \right\rceil$$

, senza i quali non è possibile dichiarare la correttezza del gossiping. Una volta ricevuti i pesi, questi vengono aggregati e faranno parte del prossimo round. Questo assicura quindi che, anche se un nodo non riceve direttamente i pesi di tutti gli altri nodi, li riceverà almeno in maniera indiretta, in quanto questi saranno incorporati nei modelli dei peers che riceverà nei round successivi.

2.3 Training Modello AI

2.3.1 Dati di training

Si è scelto di utilizzare il dataset di sentiment analysis di Twitter *sent140*. Il json contenente il dataset utilizzato durante l'addestramento è stato generato tramite lo script fornito nel README di *sent140* lanciato con il seguente comando:

```
./preprocess.sh -s niid --sf 0.05 -k 3 -t sample
```

I dati sono non identicamente distribuiti, per simulare uno scenario realistico in cui i dati dei singoli client sono disomogenei.

La struttura simula naturalmente più utenti per nodo client, perché i dati sono organizzati in:

- una lista di utenti;
- una lista con il numero di samples per ogni utente;
- quarantamila tweet classificati in base all'utente originario;

2.3.2 Modello

I nodi messi a disposizione su AWS hanno capacità di calcolo limitata alla sola CPU, per cui non è possibile compiere operazioni di addestramento computazionalmente troppo costose come per un

transformer o di una rete neurale ricorrente (RNN). È altresì necessario che il modello permetta lo scambio di pesi per l'addestramento, per cui il modello deve essere una rete neurale.

Si è quindi scelto un approccio ibrido:

- (1) Transformer di embedding preaddestrato;
- (2) Rete neurale come testa di classificazione.

Embedding

L'embedding per ogni tweet viene generato da un transformer già addestrato. Il modello utilizzato ha una dimensione di circa 300 megabyte, grazie a una tecnica chiamata *knowledge distillation*. Un modello più piccolo è stato addestrato in modo che replicasse il più possibile la distribuzione del modello più grande. La funzione di perdita della *knowledge distillation* è infatti minimizzata se il comportamento del modello "studente" tende a quello dell'"insegnante":

$$\mathcal{L}_{CE} = - \sum_i (p_i^{teacher} \cdot \log p_i^{student})$$

Il modello originale si basa su BERT, un tipo di transformer in cui l'obiettivo dell'addestramento è predire dei token nascosti all'interno della frase, mentre i modelli di tipo GPT sono addestrati a predire la parola successiva dato il contesto delle precedenti.

Ogni tweet viene prima tokenizzato nella funzione `prepare_dataset`. Nel forward pass questa tokenizzazione viene passata al modello BERT che restituisce il processamento di ogni token del tweet. Viene estratto solo il token iniziale [CLS] perché è un vettore riassuntivo dell'intera frase. Grazie al meccanismo di self-attention bidirezionale, ogni token ha scambiato informazioni con tutti gli altri. In questo modo, il token [CLS] ha al suo interno informazioni su tutti gli elementi della sequenza. Questo utilizzo è suggerito anche nel paper introduttivo su BERT [1]¹.

Rete neurale

Viene addestrato come task solo la classificazione dell'embedding di un tweet, con una rete neurale che riduce gradualmente gli elementi dell'embedding di un ordine di grandezza di 4 (128, 32, 2). Questa scelta permette di mantenere un buon livello di prestazioni² mantenendo il numero di pesi nella rete neurale limitato³.

I livelli nascosti della rete neurale sono caratterizzati dalla funzione di attivazione ReLU per ridurre il rischio di vanishing gradients, e da un livello di Dropout come tecnica di regolarizzazione per prevenire l'overfitting. Per l'iperparametro del Dropout sono state testate come configurazioni 0.1, 0.3 e 0.5. Il valore intermedio è risultato il caso migliore rispetto all'accuracy. Come funzione di classificazione sarebbe sufficiente la sola funzione Sigmoid, dove l'output finale del modello è la probabilità che il sentiment sia positivo. Vorrei sottolineare che, per motivi implementativi, è stata utilizzata la Softmax.

¹In 3.2 Fine-tuning BERT: "the [CLS] representation is fed into an output layer for classification, such as entailment or sentiment analysis."

²i test con l'accuracy in locale si aggirano sul 70%

³testando con un livello iniziale da 512 a 128 elementi, la dimensione della rete neurale arriva a 1,8 megabyte, mentre nella rete neurale finale si aggira sui 400 kilobyte

2.3.3 Addestramento modello

Per l'addestramento del modello in `train_local` si è fatto ricorso della funzione `CrossEntropyLoss` come funzione di perdita.

Quest'ultima riceve in input i logit della classificazione, calcola le probabilità con la funzione `Softmax` e su queste ultime la loss. È un meccanismo che funziona per sistemi multiclasse e quindi anche nel caso binario dove l'output dell'ultimo livello sono due logit.

Per la gestione del learning rate si è invece fatto ricorso ad Adam. È il sistema più efficace perché applica un learning rate diverso per ogni parametro della rete. Questo approccio consente di convergere meglio verso un buon risultato in un numero minore di epoche. Questo è molto importante nel nostro caso dove il numero di epoche è molto basso. A riguardo vorrei menzionare che inizialmente è stato introdotto un meccanismo di early stopping, ma si è scelto di rimuoverlo sia per il numero esiguo di epoche che per la variabilità nel tempo di calcolo di un'epoca.

Si sono riscontrate abitualmente differenze significative tra la velocità di esecuzione tra i nodi nello stesso addestramento, con nodi più lenti del triplo rispetto ai più veloci. Questo comporta che quando riceviamo i pesi degli altri nodi ne riceviamo solo una parte nel tempo di attesa massimo. Rendere quest'ultimo pari al tempo di esecuzione massimo incrementerebbe in modo significativo i tempi di addestramento. Per mitigare i danni di questo fenomeno si è scelto di ricevere ogni round in arrivo da ogni client e poi scegliere il round finale più recente, dato il client che lo ha inviato.

Il calcolo della validation loss durante l'addestramento serve a controllare l'evoluzione del modello a ogni epoca. La valutazione finale è indicativa del fatto che il modello sia in grado di generalizzare le sue predizioni sul test set del client.

2.3.4 Tolleranza ai guasti

È stato implementato un sistema di tolleranza ai guasti per gestire il crash di uno o più client durante la fase di addestramento. Quando un nodo deve propagare i suoi pesi ai vicini inizia l'algoritmo di gossiping. A seguito dell'invio dei pesi a un vicino, se la comunicazione non va a buon fine, il nodo mittente notifica il registry. A seguito di questa notifica il registry invia un ping al nodo client che non ha risposto. Se quest'ultimo non risponde il registry crea un nuovo nodo client con l'id del nodo terminato e invia le informazioni sul nuovo nodo a tutti gli altri. Questi ultimi aggiorneranno la loro lista dei peer. Il nuovo nodo riceve la configurazione da colui che ha segnalato il guasto e poi aspetta di ricevere i modelli del round attuale, li aggrega e riparte al prossimo round. Essendo già trascorso un round, questo nodo ne avrà effettivamente saltato uno. È stato anche implementato un meccanismo per evitare la creazione di un nuovo nodo nel caso esso abbia terminato l'addestramento e si sia de-registrato correttamente. Per fare ciò il registry mantiene in memoria una mappa di nodi che hanno terminato correttamente, evitando così di creare duplicati non necessari.

Nella versione centralizzata il funzionamento è lo stesso, ma la chiamata al registry e l'invio della configurazione viene effettuata dal coordinatore centrale a seguito di un timeout di attesa dei pesi.

2.3.5 Metriche di valutazione

Per avere un quadro più generale possibile si sono calcolate l'accuracy, la precision, la recall e la curva di ROC.

2.3.6 Librerie utilizzate dal modello distribuito

- (1) json e boto3 sono servite per l'accesso originale al bucket con i dati per i client;
- (2) sklearn contiene la funzione `train_test_split` per la divisione tra training e test set;
- (3) transformers per ottenere il modello distilBert che restituisce l'embedding dei tweet;
- (4) torch per implementare il modello;
- (5) grpc per la comunicazione tra nodi e col registry.
- (6) AWS SDK for Go per la comunicazione tra il service registry e AWS.

3 Sistema Centralizzato

Mentre nel sistema decentralizzato ogni nodo distribuisce i suoi pesi a fine round tramite l'algoritmo di gossiping, nel sistema centralizzato abbiamo un coordinatore dedicato all'aggregazione dei pesi. In questo contesto i nodi client che partecipano all'addestramento sono definiti *nodi worker*, per distinguerli dal nodo coordinatore, che non partecipa attivamente all'addestramento ma solo all'aggregazione.

I client ad ogni round dopo il training locale inviano i pesi al coordinatore. Quest'ultimo aspetta i pesi di tutti gli altri nodi fino a un tempo limite, poi esegue la FedAvg e spedisce il modello aggiornato a tutti i nodi worker.

Il coordinatore aspetta di ricevere i pesi da tutti, ai nodi che non inviano in tempo i loro pesi invia un ping. Se non riceve una risposta utilizza il meccanismo di failure detection per creare dei nodi sostitutivi.

Il coordinatore invia il segnale di inizio training ai nodi worker, con le variabili d'ambiente specifiche a ognuno di essi. Nel decentralizzato questo veniva effettuato dal nodo che iniziava il training.

4 Osservazioni

Si sono testati i casi con 5, 20 e 50 nodi per monitorare la scalabilità del , con una base di 4 round da 3 epoche. Nel caso con 5 nodi si sono svolti test aggiuntivi al variare del numero di round (4 e 8) e del numero di epoche per round (3 e 6). Le metriche calcolate sul test set sono rappresentate nel grafico "Confronto Metriche Finali per Nodo e Media", mentre i tempi di addestramento locale e di trasferimento dei pesi sono rappresentati nel grafico "Tempi Medi per Round per Nodo".

Si è anche monitorato in fase di addestramento la loss sul training set e la validation accuracy. La minimizzazione della loss è esemplificativa del processo di addestramento, perché il modello viene addestrato solo sul training set. La validation accuracy è invece indicativa di quanto il modello sia capace di generalizzare su dati che non ha mai visto prima dell'evaluation finale. Di entrambe queste due metriche mostriamo solo un esempio indicativo nel caso standard.

4.1 Caso decentralizzato

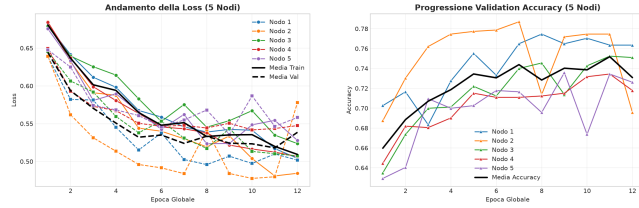


Figure 1: 5 nodi, 4 round e 3 epoche, loss e validation

Ovviamente la loss sul training set migliora sempre, mentre la validation accuracy si stabilizza al 73%.

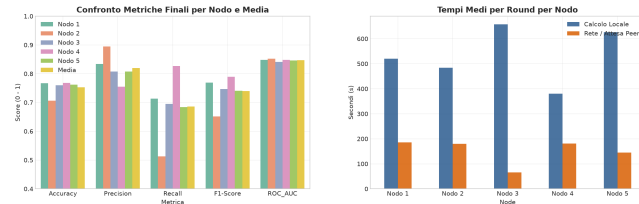


Figure 2: 5 nodi, 4 round e 3 epoche, metriche e tempi

Nell'evaluation finale possiamo osservare un comportamento anomalo nei nodi 2 e 4. Il nodo 2 ha una precision più elevata ma una recall molto più bassa. Il nodo 4 ha una precision leggermente più bassa ma una recall molto più alta. Queste caratteristiche si riflettono sull'accuracy, dove per il nodo 4 è superiore alla media, mentre per il nodo 2 è inferiore alla media. Il nodo 4 è quindi capace di riconoscere molto bene sia i tweet con sentiment positivo che negativo, mentre il nodo 2 sbaglia molto spesso a classificare i tweet con sentiment negativo. In conclusione, il nodo 2 tende a classificare i tweet come positivi, ciò ci fornisce una buona precision, ma una scarsa recall a causa di tutti i falsi positivi erroneamente catalogati.

Per i tempi medi possiamo osservare come anche il solo addestramento locale può impiegare tempi molto diversi tra ogni nodo, infatti il nodo 4 impiega metà del tempo rispetto al nodo 3. A causa di questa differenza di tempo, la maggioranza dei nodi aspetta fino al tempo limite (180 secondi) per ricevere quanti più pesi possibile, il che porta almeno un peer a non partecipare alla loro FedAvg. Il comportamento migliore del nodo 4 rispetto agli altri nodi può essere spiegato dal suo breve tempo di addestramento e dal suo tempo di ricezione dei pesi degli altri peer. Ciò gli permette di ricevere più pesi degli altri nodi fornendoci migliori risultati.

Le prestazioni del sistema sono più uniformi e si attestano tra il 75% e l'80%. Nella precision il nodo 1 ha prestazioni migliori, mentre nella recall è il nodo 2 ad avere prestazioni migliori. Si può infatti osservare che l'F1-Score dei nodi 1 e 2 è più alto rispetto agli altri nodi.

Salvo questi due casi le prestazioni sono molto simili, per cui possiamo in prima istanza supporre che l'utilizzo di più epoche ha portato a un comportamento più stabile dei modelli, poiché il contributo del dataset locale è preponderante rispetto ai pesi degli altri nodi.

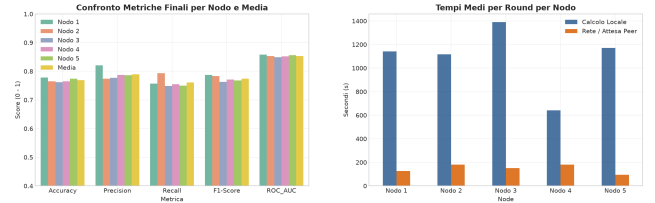


Figure 3: 5 nodi, 4 round e 6 epoche, metriche e tempi

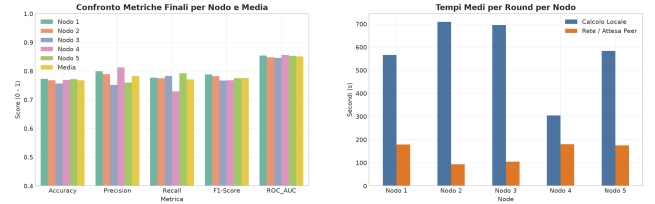


Figure 4: 5 nodi, 8 round e 3 epoche, metriche e tempi

Le prestazioni su precision e recall sono meno stabili del caso con più epoche per round aumenta l'influenza degli altri peer. Essendo il tempo di arrivo dei pesi variabile, alcuni nodi non ricevono tutti i pesi che si aspettano. Questi ultimi avranno delle prestazioni inferiori, per questo le metriche sono meno stabili del caso in cui le stesse epoche sono raggruppate in meno round.

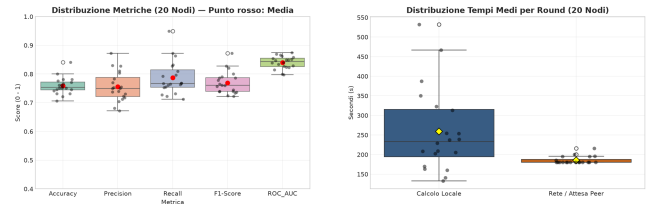


Figure 5: 20 nodi, 4 round e 3 epoche, metriche e tempi

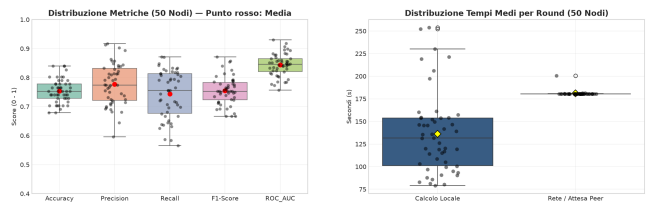


Figure 6: 50 nodi, 4 round e 3 epoche, metriche e tempi

Nei casi con 20 e 50 nodi le barre sono sostituite dai pallini grigi e la media è rappresentata nel grafico a sinistra da un pallino rosso, a destra da un rombo giallo.

Possiamo osservare una variabilità maggiore tra i nodi. Inoltre, la recall media è più alta della precision media, al contrario di ciò che è avvenuto in tutti i casi con 5 nodi. Questo singolo esempio

non rappresenta un trend evidente, perché nel caso con 50 nodi la precision torna a superare la recall. In generale si riscontra una maggiore varianza tra i modelli nel sistema, è quindi meno utile considerare le statistiche aggregate.

Un comportamento anomalo visibile in entrambi i casi è che il tempo di attesa dei peer può essere superiore al limite massimo. Questo perché nessun nodo riceve i pesi da tutti i nodi e, con un numero elevato di pesi, il tempo di processamento è significativo.

4.2 Confronto con caso centralizzato

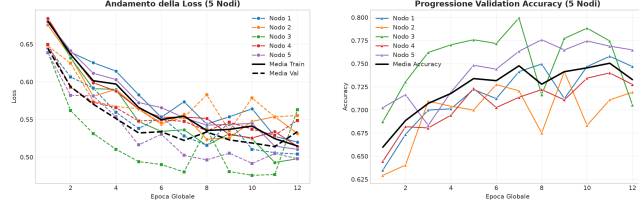


Figure 7: 5 nodi, 4 round e 3 epoche, loss e validation

Ovviamente la loss sul training set migliora sempre, mentre la validation accuracy si stabilizza al 73%.

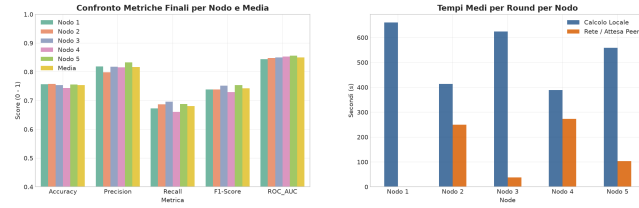


Figure 8: 5 nodi, 4 round e 3 epoche, metriche e tempi

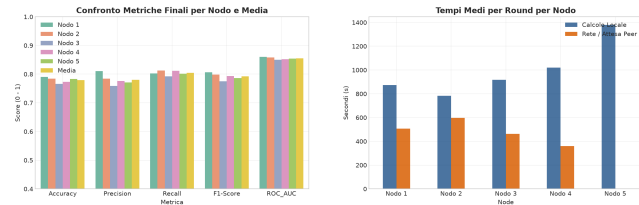


Figure 9: 5 nodi, 4 round e 6 epoche, metriche e tempi

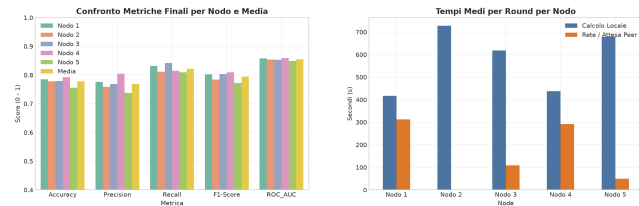


Figure 10: 5 nodi, 8 round e 3 epoche, metriche e tempi

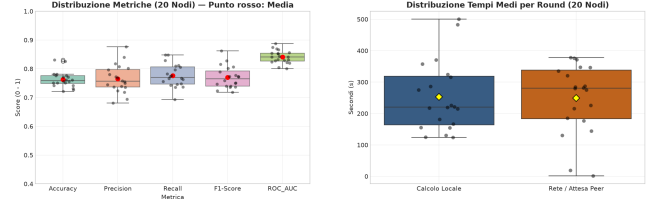


Figure 11: 20 nodi, 4 round e 3 epoche, metriche e tempi

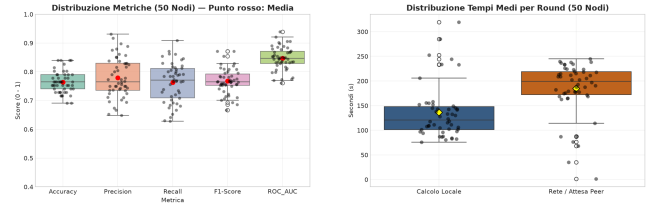


Figure 12: 20 nodi, 4 round e 3 epoche, metriche e tempi

Type	Accuracy	Precision	Recall	F1-Score
5,4,3,D	0.7527	0.8197	0.6864	0.7396
5,4,3,C	0.7537	0.8167	0.6809	0.7426
5,4,6,D	0.7688	0.7892	0.7608	0.7745
5,4,6,C	0.7796	0.7802	0.8044	0.7920
5,8,3,D	0.7681	0.7825	0.7713	0.7763
5,8,3,C	0.7783	0.7694	0.8219	0.7946
20,4,3,D	0.7587	0.7560	0.7869	0.7694
20,4,3,C	0.7632	0.7647	0.7766	0.7699
50,4,3,D	0.7543	0.7776	0.7437	0.7553
50,4,3,C	0.7639	0.7796	0.7635	0.7679

Table 1: Media metriche nei diversi scenari

Messe a confronto queste due tipologie di sistemi, l'Accuracy e l'F1-Score sono migliori nel caso centralizzato. Questo dipende dal fatto che il modello nel caso centralizzato è uniforme grazie alla FedAvg svolta dal controllore. Ricordiamo ovviamente che lo svantaggio principale per un sistema distribuito è avere un single point of failure, perché se il controllore fallisce non vi è un meccanismo di failure detection e di ripristino, nonché il bottleneck causato da un solo componente che deve ricevere tutti i pesi. Per finire, la richiesta di un hardware capace di supportare queste specifiche richiede inevitabilmente una sorta di entità centrale che lo possegga. D'altra parte, nel caso decentralizzato, nonostante l'utilizzo dell'algoritmo di gossiping, il traffico di rete per lo scambio dei pesi cresce esponenzialmente col numero di clienti.

Un'importante caratteristica da osservare è che, avendo un solo componente centralizzato che riceve tutti i pesi, il nodo più lento è l'ultimo a trasmetterli. Per questo, dai tempi medi di attesa dei peer, questo nodo si distingue per un tempo di attesa nullo.

5 Conclusioni

I risultati delle osservazioni e le considerazioni qui esposte hanno valenza limitata dalle dimensioni del dataset, dalle capacità di adattamento della rete neurale stessa e dalle caratteristiche dei nodi coinvolti, come il numero e la capacità di calcolo.

Il dataset è stato limitato a circa 40 mila sample di tweet. I container che simulavano i nodi hanno avuto a disposizione solo la CPU essendo la GPU non supportata da Fargate. Solo per l'addestramento con 1000 sample (nei casi con 5 nodi), il tempo di addestramento per round (3 epoche) si aggira sui 500 secondi, oltre 8 minuti. Questo implica che il tempo medio per completare un addestramento nel caso con 4 round è di almeno 30 minuti. Per questo i dati visibili al singolo nodo sono stati limitati a 1000 sample.

Inoltre, non ci è stato possibile testare troppe configurazioni della rete neurale. Inizialmente la rete era composta solo da un livello di output che restituiva le due classi, in questa fase infatti l'accuracy si attestava sul 55%, quasi come compiere una scelta casuale. In seguito si è testato dal livello di classificazione con un approccio simile a LeNet5, con una riduzione di parametri da ogni livello, dalla dimensione di output di distilBert a 512, 128, 32 e 2. Questo ha determinato il modello da 1,8 milioni di parametri, con accuracy che

si attestava in prima istanza sul 75%. Togliendo il livello iniziale di 512 parametri in input, e quindi avendo solo da distilBert a 128 e così via, l'accuracy si attestava sul 73%, quindi aver rimosso quel livello non ha impattato significativamente le capacità di generalizzazione del modello. Non è stato testato il caso in cui si avessero più livelli da 128 parametri. I test presentati nelle osservazioni sono stati compiuti solo con il modello finale, perché lo svolgimento di tutti i test una sola volta comporta una spesa di circa 30\$ di credito sul corso in Canvas.

In conclusione, lo svolgimento del test distribuito non determina una degradazione delle prestazioni troppo significativa rispetto allo svolgimento del modello in locale, ma un numero significativo di nodi nel caso decentralizzato determina grande variabilità nelle capacità di generalizzazione dei modelli dei singoli nodi.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL] <https://arxiv.org/abs/1810.04805>
- [2] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. <http://arxiv.org/abs/1602.05629>